Intro
Components
- Button
- Checkbox
- OptionButton
- GroupBox
- TextBox
- Slider
- Dropdown
- Window
  - Title Bar
  - Window contents
  - Status Bar
- TreeView
- Tabs
- TableView
- Progress Indicator
- Field borders
Issues, Contributing, etc.

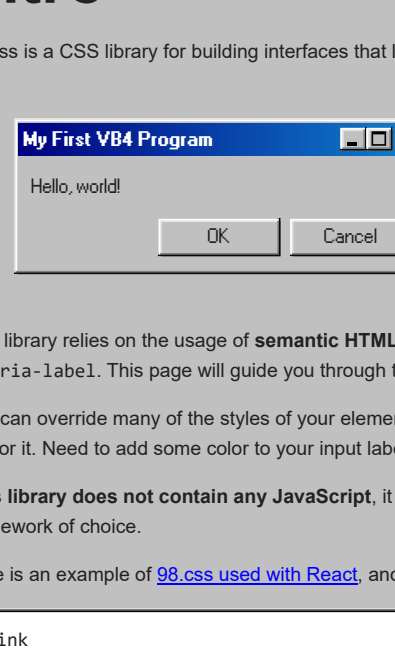
98.css

A design system for building faithful recreations of old UIs.

npm v0.1.21 | gzip 4.35 KB

Intro

98.css is a CSS library for building interfaces that look like Windows 98. See more on GitHub.



This library relies on the usage of **semantic HTML**. To make a button, you'll need to use a `<button>`. Input elements require labels. Icon buttons rely on `aria-label`. This page will guide you through that process, but accessibility is a primary goal of this project.

You can override many of the styles of your elements while maintaining the appearance provided by this library. Need more padding on your buttons? Go for it. Need to add some color to your input labels? Be our guest.

This library does not contain any JavaScript, it merely styles your HTML with some CSS. This means 98.css is compatible with your frontend framework of choice.

Here is an example of 98.css used with React, and an example with vanilla JavaScript. The fastest way to use 98.css is to import it from unpkg.

```
<link
  rel="stylesheet"
  href="https://unpkg.com/98.css"
>
```

You can install 98.css from the GitHub releases page, or from npm.

```
npm install 98.css
```

Components

Button

> A *command button*, also referred to as a push button, is a control that causes the application to perform some action when the user clicks it.
>
> — Microsoft Windows User Experience p. 160

A standard button measures 75px wide and 23px tall, with a raised outer and inner border. They are given 12px of horizontal padding by default.

Click me | Submit | Reset

Show code

```
<button>Click me</button>
<input type="submit" />
<input type="reset" />
```

You can add the class `default` to any button to apply additional styling, useful when communicating to the user what default action would happen in the active window if the `Enter` key was pressed on Windows 98.

OK

Show code

```
<button class="default">OK</button>
```

When buttons are clicked, the raised borders become sunken. The following button is simulated to be in the pressed (active) state.

I am being pressed

Show code

```
<button>I am being pressed</button>
```

Disabled buttons maintain the same raised border, but have a "washed out" appearance in their label.

I cannot be clicked

Show code

```
<button disabled>I cannot be clicked</button>
```

Button focus is communicated with a dotted border, set 4px within the contents of the button. The following example is simulated to be focused.

I am focused

Show code

```
<button>I am focused</button>
```

Checkbox

> A *check box* represents an independent or non-exclusive choice.
>
> — Microsoft Windows User Experience p. 167

Checkboxes are represented with a sunken panel, populated with a "check" icon when selected, next to a label indicating the choice.

Note: You **must** include a corresponding label **after** your checkbox, using the `<label>` element with a `for` attribute pointed at the `id` of your input. This ensures the checkbox is easy to use with assistive technologies, on top of ensuring a good user experience for all (navigating with the tab key, being able to click the entire label to select the box).

☐ This is a checkbox

Show code

```
<input type="checkbox" id="example1">
<label for="example1">This is a checkbox</label>
```

Checkboxes can be selected and disabled with the standard `checked` and `disabled` attributes.

When grouping inputs, wrap each input in a container with the `field-row` class. This ensures a consistent spacing between inputs.

☑ I am checked
☐ I am inactive
☑ I am inactive but still checked

Show code

```
<div class="field-row">
  <input checked type="checkbox" id="example2">
  <label for="example2">I am checked</label>
</div>
<div class="field-row">
  <input disabled type="checkbox" id="example3">
  <label for="example3">I am inactive</label>
</div>
<div class="field-row">
  <input checked disabled type="checkbox" id="example4">
  <label for="example4">I am inactive but still checked</label>
</div>
```

OptionButton

> An *option button*, also referred to as a radio button, represents a single choice within a limited set of mutually exclusive choices. That is, the user can choose only one set of options.
>
> — Microsoft Windows User Experience p. 164

Option buttons can be used via the `radio` type on an input element.

Option buttons can be grouped by specifying a shared `name` attribute on each input. Just as before: when grouping inputs, wrap each input in a container with the `field-row` class to ensure a consistent spacing between inputs.

○ Yes
○ No

Show code

```
<div class="field-row">
  <input id="radio5" type="radio" name="first-example">
  <label for="radio5">Yes</label>
</div>
<div class="field-row">
  <input id="radio6" type="radio" name="first-example">
  <label for="radio6">No</label>
</div>
```

Option buttons can also be `checked` and `disabled` with their corresponding HTML attributes.

○ Peanut butter should be smooth
● I understand why people like crunchy peanut butter
○ Crunchy peanut butter is good

Show code

```
<div class="field-row">
  <input id="radio7" type="radio" name="second-example">
  <label for="radio7">Peanut butter should be smooth</label>
</div>
<div class="field-row">
  <input checked disabled id="radio8" type="radio" name="second-example">
  <label for="radio8">I understand why people like crunchy peanut butter</label>
</div>
<div class="field-row">
  <input disabled id="radio9" type="radio" name="second-example">
  <label for="radio9">Crunchy peanut butter is good</label>
</div>
```

GroupBox

> A *group box* is a special control you can use to organize a set of controls. A group box is a rectangular frame with an optional label that surrounds a set of controls.
>
> — Microsoft Windows User Experience p. 189

A group box can be used by wrapping your elements with the `fieldset` tag. It contains a sunken outer border and a raised inner border, resembling an engraved box around your controls.

Select one:
○ Diners
○ Drive-Ins
○ Dives

Show code

```
<fieldset>
  <div class="field-row">Select one:</div>
  <div class="field-row">
    <input id="radio10" type="radio" name="fieldset-example">
    <label for="radio10">Diners</label>
  </div>
  <div class="field-row">
    <input id="radio11" type="radio" name="fieldset-example">
    <label for="radio11">Drive-Ins</label>
  </div>
  <div class="field-row">
    <input id="radio12" type="radio" name="fieldset-example">
    <label for="radio12">Dives</label>
  </div>
</fieldset>
```

You can provide your group with a label by placing a `legend` element within the `fieldset`.

Today's mood
○ Claire Saffitz
○ Brad Leone
○ Chris Morocco
○ Carla Lalli Music

Show code

```
<fieldset>
  <legend>Today's mood</legend>
  <div class="field-row">
    <input id="radio13" type="radio" name="fieldset-example2">
    <label for="radio13">Claire Saffitz</label>
  </div>
  <div class="field-row">
    <input id="radio14" type="radio" name="fieldset-example2">
    <label for="radio14">Brad Leone</label>
  </div>
  <div class="field-row">
    <input id="radio15" type="radio" name="fieldset-example2">
    <label for="radio15">Chris Morocco</label>
  </div>
  <div class="field-row">
    <input id="radio16" type="radio" name="fieldset-example2">
    <label for="radio16">Carla Lalli Music</label>
  </div>
</fieldset>
```

TextBox

> A *text box* (also referred to as an edit control) is a rectangular control where the user enters or edits text. It can be defined to support a single line or multiple lines of text.
>
> — Microsoft Windows User Experience p. 181

Text boxes can rendered by specifying a `text` type on an `input` element. As with checkboxes and radio buttons, you should provide a corresponding label with a properly set `for` attribute, and wrap both in a container with the `field-row` class.

Occupation

Show code

```
<div class="field-row">
  <label for="text17">Occupation</label>
  <input id="text17" type="text" />
</div>
```

Additionally, you can make use of the `field-row-stacked` class to position your label above the input instead of beside it.

Address (Line 1)

Address (Line 2)

Show code

```
<div class="field-row-stacked" style="width: 200px">
  <label for="text18">Address (Line 1)</label>
  <input id="text18" type="text" />
</div>
<div class="field-row-stacked" style="width: 200px">
  <label for="text19">Address (Line 2)</label>
  <input id="text19" type="text" />
</div>
```

To support multiple lines in the user's input, use the `textarea` element instead.

Additional notes

Show code

```
<div class="field-row-stacked" style="width: 200px">
  <label for="text20">Additional notes</label>
  <textarea id="text20" rows="8"></textarea>
</div>
```

Text boxes can also be disabled and have value with their corresponding HTML attributes.

Favorite color Windows Green

Show code

```
<div class="field-row">
  <label for="text21">Favorite color</label>
  <input id="text21" disabled type="text" value="Windows Green"/>
</div>
```

Slider

> A *slider*, sometimes called a trackbar control, consists of a bar that defines the extent or range of the adjustment and an indicator that shows the current value for the control...
>
> — Microsoft Windows User Experience p. 146

Sliders can rendered by specifying a `range` type on an `input` element.

Volume: Low High

Show code

```
<div class="field-row" style="width: 300px">
  <label for="range22">Volume:</label>
  <label for="range23">Low</label>
  <input id="range23" type="range" min="1" max="11" value="5" />
  <label for="range24">High</label>
</div>
```

You can make use of the `has-box-indicator` class replace the default indicator with a box indicator, furthermore the slider can be wrapped with a `div` using `is-vertical` to display the input vertically.

Note: To change the length of a vertical slider, the `input` width and `div` height.

Cowbell

Show code

```
<div class="field-row">
  <label for="range25">Cowbell</label>
  <div class="is-vertical">
    <input id="range25" class="has-box-indicator" type="range" min="1" max="3" step="1" value="2" />
  </div>
</div>
```

Dropdown

> A *drop-down list box* allows the selection of only a single item from a list. In its closed state, the control displays the current value for the control. The user opens the list to change the value.
>
> — Microsoft Windows User Experience p. 175

Dropdowns can be rendered by using the `select` and `option` elements.

5 - Incredible!

Show code

```
<select>
  <option>5 - Incredible!</option>
  <option>4 - Great!</option>
  <option>3 - Pretty good</option>
  <option>2 - Not so great</option>
  <option>1 - Unfortunate</option>
</select>
```

By default, the first option will be selected. You can change this by giving one of your `option` elements the `selected` attribute.

3 - Pretty good

Show code

```
<select>
  <option>5 - Incredible!</option>
  <option>4 - Great!</option>
  <option selected>3 - Pretty good</option>
  <option>2 - Not so great</option>
  <option>1 - Unfortunate</option>
</select>
```

Window

The following components illustrate how to build complete windows using 98.css.

Title Bar

> At the top edge of the window, inside its border, is the title bar (also referred to as the caption or caption bar), which extends across the width of the window. The title bar identifies the contents of the window.
>
> — Microsoft Windows User Experience p. 118

> Include command buttons associated with the common commands of the primary window in the title bar. These buttons act as shortcuts to specific window commands.
>
> — Microsoft Windows User Experience p. 122

You can build a complete title bar by making use of three classes, `title-bar`, `title-bar-text`, and `title-bar-controls`.

A Title Bar

Show code

```
<div class="title-bar">
  <div class="title-bar-text">A Title Bar</div>
  <div class="title-bar-controls">
    <button aria-label="Close"></button>
  </div>
</div>
```

We make use of `aria-label` to render the Close button, to let assistive technologies know the intent of this button. You may also use "Minimize", "Maximize", "Restore" and "Help" like so:

A Title Bar

A Maximized Title Bar

A Helpful Bar

Show code

```
<div class="title-bar">
  <div class="title-bar-text">A Title Bar</div>
  <div class="title-bar-controls">
    <button aria-label="Minimize"></button>
    <button aria-label="Maximize"></button>
    <button aria-label="Close"></button>
  </div>
</div>

<br />

<div class="title-bar">
  <div class="title-bar-text">A Maximized Title Bar</div>
  <div class="title-bar-controls">
    <button aria-label="Minimize"></button>
    <button aria-label="Restore"></button>
    <button aria-label="Close"></button>
  </div>
</div>

<br />

<div class="title-bar">
  <div class="title-bar-text">A Helpful Bar</div>
  <div class="title-bar-controls">
    <button aria-label="Help"></button>
    <button aria-label="Close"></button>
  </div>
</div>
```

Each `aria-label` also has a corresponding styling class to render the title bar buttons, to let the `aria-label` text be in other languages without causing rendering, accessibility, or localization issues.

A Title Bar using Button Styling Classes

A Maximized Title Bar using Button Styling Classes

A Helpful Bar using Button Styling Classes

Show code

```
<div class="title-bar">
  <div class="title-bar-text">A Title Bar using Button Styling Classes</div>
  <div class="title-bar-controls">
    <button aria-label="Any Text" class="minimize"></button>
    <button aria-label="Any Text" class="maximize"></button>
    <button aria-label="Any Text" class="close"></button>
  </div>
</div>

<br />

<div class="title-bar">
  <div class="title-bar-text">A Maximized Title Bar using Button Styling Classes</div>
  <div class="title-bar-controls">
    <button aria-label="Any Text" class="minimize"></button>
    <button aria-label="Any Text" class="restore"></button>
    <button aria-label="Any Text" class="close"></button>
  </div>
</div>

<br />

<div class="title-bar">
  <div class="title-bar-text">A Helpful Bar using Button Styling Classes</div>
  <div class="title-bar-controls">
    <button aria-label="Any Text" class="help"></button>
    <button aria-label="Any Text" class="close"></button>
  </div>
</div>
```

Maximize buttons can be disabled, useful when making a window appear as if it cannot be maximized.

A Title Bar with Maximize disabled

Show code

```
<div class="title-bar">
  <div class="title-bar-text">A Title Bar with Maximize disabled</div>
  <div class="title-bar-controls">
    <button aria-label="Minimize"></button>
    <button aria-label="Maximize" disabled></button>
    <button aria-label="Close"></button>
  </div>
</div>
```

You can make a title bar "inactive" by adding `inactive` class, useful when making more than one window.

An inactive title bar

Show code

```
<div class="title-bar inactive">
  <div class="title-bar-text">An inactive title bar</div>
  <div class="title-bar-controls">
    <button aria-label="Close"></button>
  </div>
</div>
```

Window contents

> Every window has a boundary that defines its shape.
>
> — Microsoft Windows User Experience p. 118

To give our title bar a home, we make use of the `window` class. This provides a raised outer and inner border, as well as some padding. We can freely resize the window by specifying a width in the container style.

A Complete Window

Show code

```
<div class="window" style="width: 300px">
  <div class="title-bar">
    <div class="title-bar-text">A Complete Window</div>
    <div class="title-bar-controls">
      <button aria-label="Minimize"></button>
      <button aria-label="Maximize"></button>
      <button aria-label="Close"></button>
    </div>
  </div>
</div>
```

To draw the contents of the window, we use the `window-body` class under the title bar.

A Window With Stuff In It

There's so much room for activities!

Show code

```
<div class="window" style="width: 300px">
  <div class="title-bar">
    <div class="title-bar-text">A Window With Stuff In It</div>
    <div class="title-bar-controls">
      <button aria-label="Minimize"></button>
      <button aria-label="Maximize"></button>
      <button aria-label="Close"></button>
    </div>
  </div>
  <div class="window-body">
    <p>There's so much room for activities!</p>
  </div>
</div>
```

Status Bar

> A *status bar* is a special area within a window, typically the bottom, that displays information about the current state of what is being viewed in the window or any other contextual information, such as keyboard state.
>
> — Microsoft Windows User Experience p. 146

You can render a status bar with the `status-bar` class, and `status-bar-field` for every child text element.

A Window With A Status Bar

There are just so many possibilities:

- A Task Manager
- A Notepad
- Or even a File Explorer!

Press F1 for help | Slide 1 | CPU Usage: 14%

Show code

```
<div class="window" style="width: 320px">
  <div class="title-bar">
    <div class="title-bar-text">A Window With A Status Bar</div>
  </div>
  <div class="window-body">
<p> There are just so many possibilities:</p>
<ul>
    <li>A Task Manager</li>
    <li>A Notepad</li>
    <li>Or even a File Explorer!</li>
</ul>
  </div>
  <div class="status-bar">
    <p class="status-bar-field">Press F1 for help</p>
    <p class="status-bar-field">Slide 1</p>
    <p class="status-bar-field">CPU Usage: 14%</p>
  </div>
</div>
```

TreeView

> A *tree view control* is a special list box control that displays a set of objects as an indented outline based on their logical hierarchical relationship.
>
> — Microsoft Windows User Experience p. 178

To render a tree view, use an `ul` element with the `tree-view` class. The children of this list (`li` elements), can contain whatever you'd like.

We can put
✨ **Whatever** ✨
We want in here

Show code

```
<ul class="tree-view">
  <li>We can put</li>
  <li><strong style="color: purple">✨ Whatever ✨</strong></li>
  <li>We want in here</li>
</ul>
```

To make this a tree, we can nest further `ul` elements (no class needed on these). This will provide them with a nice dotted border and indentation to illustrate the structure of the tree.

To create expandable sections, wrap child lists inside of `details` elements.

Table of Contents
What is web development?
CSS
- Selectors
- Specificity
- Properties

⊟ JavaScript
- Avoid at all costs
- ⊞ Unless

HTML
Special Thanks

Show code

```
<ul class="tree-view">
  <li>Table of Contents</li>
  <li>What is web development?</li>
  <li>
    CSS
    <ul>
      <li>Selectors</li>
      <li>Specificity</li>
      <li>Properties</li>
    </ul>
  </li>
  <li>
    <details open>
      <summary>JavaScript</summary>
      <ul>
        <li>Avoid at all costs</li>
        <li>
          <details>
            <summary>Unless</summary>
            <ul>
              <li>Avoid</li>
              <li>
                <details>
                  <summary>At</summary>
                  <ul>
                    <li>Avoid</li>
                    <li>At</li>
                    <li>All</li>
                    <li>Cost</li>
                  </ul>
                </details>
              </li>
              <li>All</li>
              <li>Cost</li>
            </ul>
          </details>
        </li>
      </ul>
    </details>
  </li>
  <li>HTML</li>
  <li>Special Thanks</li>
</ul>
```

Tabs

> A *tab control* is analogous to a divider in a file cabinet or notebook. You can use this control to define multiple logical pages or sections of information within the same window.
>
> — Microsoft Windows User Experience p. 193

To render a tab list, use a `menu` element with the `[role=tablist]` attribute. The children of this menu (`li` elements), should get a `[role=tab]` attribute.

Tabs should be managed by adding custom javascript code. All you need is to add the `[aria-selected=true]` attribute to the active tab.

Hello, world!

Desktop | My computer | Control panel | Devices manager | Hardware profiles | Performance

the tab content

Show code

```
<div class="window-body">
  <p>Hello, world!</p>

  <menu role="tablist">
    <li role="tab" aria-selected="true"><a href="#tabs">Desktop</a></li>
    <li role="tab"><a href="#tabs">My computer</a></li>
    <li role="tab"><a href="#tabs">Control panel</a></li>
    <li role="tab"><a href="#tabs">Devices manager</a></li>
    <li role="tab"><a href="#tabs">Hardware profiles</a></li>
    <li role="tab"><a href="#tabs">Performance</a></li>
  </menu>
  <div class="window" role="tabpanel">
    <div class="window-body">
      <p>the tab content</p>
    </div>
  </div>
</div>
```

To create multirows tabs, add a `multirows` class to the `menu` tag.

Hello, world!

Desktop | My computer | Control panel | Devices manager | Hardware profiles | Performance
Users | Network | Programs | Services | Resources | Advanced

the tab content

Show code

```
<div class="window-body">
  <p>Hello, world!</p>

  <menu role="tablist" class="multirows">
    <li role="tab"><a href="#tabs">Desktop</a></li>
    <li role="tab"><a href="#tabs">My computer</a></li>
    <li role="tab"><a href="#tabs">Control panel</a></li>
    <li role="tab"><a href="#tabs">Devices manager</a></li>
    <li role="tab"><a href="#tabs">Hardware profiles</a></li>
    <li role="tab"><a href="#tabs">Performance</a></li>
  </menu>
  <menu role="tablist" class="multirows">
    <li role="tab"><a href="#tabs">Users</a></li>
    <li role="tab"><a href="#tabs">Network</a></li>
    <li role="tab"><a href="#tabs">Programs</a></li>
    <li role="tab"><a href="#tabs">Services</a></li>
    <li role="tab"><a href="#tabs">Resources</a></li>
    <li role="tab"><a href="#tabs">Advanced</a></li>
  </menu>
  <div class="window" role="tabpanel">
    <div class="window-body">
      <p>the tab content</p>
    </div>
  </div>
</div>
```

TableView

To render a table view, use a table element. Wrap it with a div element with `sunken-panel` class to provide proper border and overflow container.

With a bit of extra scripting you can make table view interactive. Give `interactive` class to table element to show pointer cursor when hovering over body rows. Table rows can be given `highlighted` class to appear selected.

Name	Version	Compa
SQL Server	3.70.06.23	Microsc
SQL Server	3.70.06.23	Microsc
SQL Server	3.70.06.23	Microsc
SQL Server	3.70.06.23	Microsc
SQL Server	3.70.06.23	Microsc
SQL Server	3.70.06.23	Microsc

Show code

```
<div class="sunken-panel" style="height: 120px; width: 240px;">
  <table class="interactive">
    <thead>
      <tr>
        <th>Name</th>
        <th>Version</th>
        <th>Company</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>MySQL ODBC 3.51 Driver</td>
        <td>3.51.11.00</td>
        <td>MySQL AB</td>
      </tr>
      <tr>
        <td>SQL Server</td>
        <td>3.70.06.23</td>
        <td>Microsoft Corporation</td>
      </tr>
      <tr>
        <td>SQL Server</td>
        <td>3.70.06.23</td>
        <td>Microsoft Corporation</td>
      </tr>
      <tr>
        <td>SQL Server</td>
        <td>3.70.06.23</td>
        <td>Microsoft Corporation</td>
      </tr>
      <tr>
        <td>SQL Server</td>
        <td>3.70.06.23</td>
        <td>Microsoft Corporation</td>
      </tr>
      <tr>
        <td>SQL Server</td>
        <td>3.70.06.23</td>
        <td>Microsoft Corporation</td>
      </tr>
      <tr>
        <td>SQL Server</td>
        <td>3.70.06.23</td>
        <td>Microsoft Corporation</td>
      </tr>
      <tr>
        <td>SQL Server</td>
        <td>3.70.06.23</td>
        <td>Microsoft Corporation</td>
      </tr>
      <tr>
        <td>SQL Server</td>
        <td>3.70.06.23</td>
        <td>Microsoft Corporation</td>
      </tr>
    </tbody>
  </table>
</div>
<script>
  document.querySelectorAll('table.interactive').forEach(element => {
    element.addEventListener('click', (event) => {
      const highlightedClass = 'highlighted';
      const isRow = element => element.tagName === 'TR' && element.parentElement.tagName === 'TBODY';
      const newlySelectedRow = event.composedPath().find(isRow);
      const previouslySelectedRow = Array.from(newlySelectedRow.parentElement.children).filter(isRow).find(element => element.classList.contains(highlightedClass));
      if(previouslySelectedRow){
        previouslySelectedRow.classList.toggle(highlightedClass);
      }

      if (newlySelectedRow) {
        newlySelectedRow.classList.toggle(highlightedClass);
      }
    })
  });
</script>
```

Progress Indicator

> You can use a *progress indicator*, also known as a *progress bar control*, to show the percentage of completion of a lengthy operation.
>
> — Microsoft Windows User Experience p. 189

There are two types of progress bars: solid and segmented. The solid version is the default. To declare a segmented bar, you should use the `segmented` class.

Show code

```
<div class="progress-indicator">
  <span class="progress-indicator-bar"  style="width: 40%;" />
</div>
```

Show code

```
<div class="progress-indicator segmented">
  <span class="progress-indicator-bar" style="width: 40%;" />
</div>
```

Field borders

> Text boxes, check boxes, drop-down list boxes, spin boxes and list boxes use the *field border style*. You can also use the style for define the work area within a window. It uses the sunken outer and sunken inner basic border styles. For most controls, the interior of the field uses the button highlight color. For text fields, such as text boxes and combo boxes, the interior uses the button face color when the field is read-only or disabled.
>
> — Microsoft Windows User Experience p. 421

> Status fields use the *status field border style*. This style uses only the sunken outer basic border style. You use the status field style in status bars and other read-only fields where the content of the file can change dynamically.
>
> — Microsoft Windows User Experience p. 422

As mentioned in these guidelines, these styles are used in other contexts than just form elements and status fields such as to indicate work areas and dynamic content. For that reason, we provide three classes for these generic usages, `field-border`, `field-border-disabled`, and `status-field-border`. These classes only define the border and background color and minimal padding, so you will typically need to at least provide some extra padding yourself.

Work area

Show code

```
<div class="field-border" style="padding: 8px">
  Work area
</div>
```

Disabled work area

Show code

```
<div class="field-border-disabled" style="padding: 8px">
  Disabled work area
</div>
```

Dynamic content

Show code

```
<div class="status-field-border" style="padding: 8px">
  Dynamic content
</div>
```

Issues, Contributing, etc.

98.css is MIT licensed.

Refer to the GitHub issues page to see bugs in my CSS or report new ones. I'd really like to see your pull requests (especially those new to open-source!) and will happily provide code review. 98.css is a fun, silly project and I'd like to make it a fun place to build your open-source muscle.

Thank you for checking my little project out, I hope it brought you some joy today. Consider starring/following along on GitHub and maybe subscribing to more fun things on my twitter. 👋